RAPPORT DE STAGE ACADÉMIQUE



Conception et réalisation de E-biessa

Place de marché intelligente de pièces automobiles d'occasion

Réalisé par	Jasser Noomani
Domaine	Génie logiciel et intelligence artificielle
Nature	Projet académique de fin de stage
Année universitaire	2025-2026

Une plateforme sécurisée, explicable et centrée sur la compatibilité véhicule

Remerciements

Je tiens à remercier toutes les personnes qui ont contribué, directement ou indirectement, à la réalisation de ce projet. Leurs échanges, leurs retours et leurs conseils ont permis de transformer un cahier des charges ambitieux en une solution cohérente, testable et documentée.

Ce travail a également été l'occasion d'approfondir des domaines complémentaires : conception d'API sécurisées, développement d'interfaces Angular, modélisation de données, recherche sémantique, adaptation d'un modèle de langage et validation de règles de compatibilité automobile.

Résumé

E-biessa est une place de marché intelligente dédiée aux pièces automobiles d'occasion en Tunisie. La plateforme réunit trois profils - acheteur, vendeur et administrateur - autour d'un parcours complet : création de compte, gestion du profil, publication d'annonces illustrées, recherche multicritère, contact, messagerie instantanée, modération et suivi de la réputation.

L'originalité de la solution réside dans une architecture hybride. Spring Boot garantit les transactions, les droits d'accès et la cohérence du catalogue. Angular fournit une interface responsive en modes clair et sombre. Un service FastAPI réalise l'indexation vectorielle et la recherche sémantique. Un modèle Qwen 2.5 7B, adapté localement par QLoRA, extrait les contraintes véhicule, carburant, position et type de pièce à partir de requêtes en français, en anglais ou en arabe tunisien. Les résultats sont ensuite contrôlés par des règles déterministes afin d'éviter toute compatibilité inventée.

La version finale intègre aussi des recommandations personnalisées explicables, une orchestration LangChain en lecture seule, une assistance VIN, une estimation indicative des prix, une identification de pièces par image et une saisie vocale. La validation distingue les règles testées automatiquement des intégrations optionnelles qui exigent des services externes. L’application constitue un démonstrateur académique ; elle ne réalise ni paiement ni commande en ligne.

Mots-clés : marketplace, pièces automobiles, Spring Boot, Angular, FastAPI, Qdrant, Qwen, QLoRA, RAG, recommandation, sécurité.

Abstract

E-biessa is an intelligent marketplace for used automotive parts in Tunisia. It supports buyers, sellers and administrators through secure account management, rich listings, multi-vehicle fitment, image handling, moderation, seller reputation, direct contact and real-time messaging.

The final solution uses Spring Boot for transactions and security, Angular for the user interface, and FastAPI for semantic retrieval and model-backed query understanding. A locally fine-tuned Qwen 2.5 7B model proposes automotive constraints, while deterministic catalogue rules remain authoritative. Explainable recommendations, read-only LangChain orchestration, VIN assistance, indicative pricing, optional CLIP image identification and editable voice input complete the academic demonstrator. Online payments and production deployment are outside its delivered scope.

Keywords: automotive marketplace, semantic search, fitment, Spring Boot, Angular, FastAPI, Qdrant, QLoRA, recommendations.

Sommaire

Liste des figures 7

Introduction générale **8**

1. Présentation du projet **9**

1.1 Contexte 9

1.2 Étude de l'existant 9

1.3 Problématique 9

1.4 Solution proposée 10

1.5 Démarche de réalisation 10

2. Analyse et spécification des besoins **11**

2.1 Identification des acteurs 11

2.2 Besoins fonctionnels 11

2.3 Besoins non fonctionnels 12

2.4 Règles métier centrales 12

2.5 Matrice de traçabilité 13

2.6 Diagrammes de cas d'utilisation 13

3. Conception **16**

3.1 Choix d'architecture 16

3.2 Architecture logique 17

3.3 Modèle de données 17

3.4 Conception de la recherche intelligente 22

3.5 Orchestration agentique finale 22

3.6 Sécurité 23

3.7 Conception de la messagerie 24

3.8 Conception de la recommandation 25

4. Réalisation **26**

4.1 Environnement de travail 26

4.2 Backend Spring Boot 26

4.3 Frontend Angular 26

4.4 Gestion complète des annonces 27

4.5 Comptes, profils et confiance 27

4.6 Contact et messagerie 27

4.7 Mise en oeuvre de l'IA . . . 28
4.8 Jeu de démonstration . . . 28
4.9 Configuration et exécution locale . . . 28
4.10 Assistance VIN et catalogue automobile . . . 29
4.11 Estimation indicative du prix . . . 29
4.12 Identification de pièces par image . . . 29
4.13 Saisie vocale . . . 30

5. Tests et validation **31**

5.1 Stratégie . . . 31
5.2 Résultats automatisés . . . 31
5.3 Évaluation du modèle de compréhension . . . 32
5.4 Validation de la recherche et des recommandations . . . 32
5.5 Recette fonctionnelle . . . 33
5.6 Limites et mesures compensatoires . . . 33
5.7 Couverture du cahier des charges . . . 34

Conclusion générale **35**

Perspectives . . . 35

Bibliographie et webographie **36**

Annexe A - Cartographie des API **37**

Annexe B - Variables de configuration . . . 37
Annexe C - Commandes de validation . . . 38
Annexe D Compléments techniques . . . 38

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
DTO	Data Transfer Object
E2E	End-to-End
JWT	JSON Web Token
LLM	Large Language Model
OEM	Original Equipment Manufacturer
QLoRA	Quantized Low-Rank Adaptation
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
SPA	Single-Page Application
UML	Unified Modeling Language
WebSocket	Canal bidirectionnel persistant entre client et serveur

Liste des figures

Figure 1 - Cas d’utilisation de consultation et d’achat

Figure 2 - Cas d’utilisation de vente et d’administration

Figure 3 - Architecture globale de la solution

Figure 4 - Classes du catalogue et des favoris

Figure 5 - Classes de communication et de réputation

Figure 6 - Classes d’identité et de personnalisation

Figure 7 - Classes de modération

Figure 8 - Chaîne de recherche intelligente et réponse ancrée

Figure 9 - Séquence simplifiée de la messagerie instantanée

Figure 10 - Recommandation personnalisée explicable

Introduction générale

Le marché de la pièce automobile d'occasion répond à un double besoin : réduire le coût de l'entretien et prolonger la durée de vie de composants encore utilisables. Pourtant, la recherche reste difficile. Une même pièce peut porter plusieurs références, convenir à plusieurs véhicules et dépendre de critères précis tels que l'année, le moteur, le carburant ou la position sur le véhicule. Une annonce incomplète ou une interprétation approximative peut donc mener à un achat inutile.

Le projet E-biessa a été conçu pour structurer ce marché et simplifier la mise en relation locale. Il ne s'agit pas seulement d'un catalogue : la plateforme associe des fonctions de place de marché, de confiance, de communication et d'intelligence artificielle. L'acheteur décrit son besoin avec ses propres mots, le système extrait les contraintes, recherche les annonces sémantiquement proches, puis vérifie la compatibilité déclarée avant d'afficher une réponse.

La problématique générale est la suivante : comment construire une place de marché automobile intuitive et riche, tout en garantissant que l'intelligence artificielle ne présente jamais une compatibilité non vérifiée ? La réponse retenue repose sur une séparation nette des responsabilités. Les données transactionnelles et l'autorisation restent dans Spring Boot et MySQL. La recherche intelligente évolue dans un service Python isolé. Les règles déterministes ont toujours priorité sur la sortie du modèle.

Le premier chapitre présente le contexte et la démarche. Le deuxième formalise les besoins et les cas d’utilisation. Le troisième détaille l’architecture, les classes et les principaux échanges. Le quatrième décrit la réalisation, y compris les aides VIN, prix, image et voix. Le cinquième distingue les résultats des tests automatisés, l’évaluation du modèle et les limites expérimentales.

1. Présentation du projet

Ce chapitre situe le besoin, les utilisateurs visés, les contraintes du domaine automobile et la démarche qui a conduit à la solution finale.

1.1 Contexte

En Tunisie, les échanges de pièces d'occasion passent souvent par des annonces généralistes, des groupes sociaux ou des contacts directs. Ces canaux favorisent la proximité, mais ils structurent peu les informations essentielles. Le titre d'une annonce peut mentionner un véhicule sans préciser la génération, le moteur ou la position. Les coordonnées du vendeur sont parfois exposées publiquement, tandis que les mécanismes de réputation et de modération restent limités.

E-biessa centralise ces échanges dans une application spécialisée. Le catalogue impose des champs cohérents, permet plusieurs véhicules compatibles et distingue la référence OEM principale des références interchangeables. Les fonctionnalités de confiance - validation des vendeurs, avis, signalements, suspension et journal d'audit - accompagnent la recherche et la transaction.

1.2 Étude de l'existant

Approche	Forces	Limites observées
Petites annonces généralistes	Audience large, publication rapide	Compatibilité peu structurée, filtres insuffisants
Groupes sociaux	Échanges directs, communauté locale	Recherche instable, données dispersées, modération variable
Catalogues professionnels	Références précises, stocks organisés	Souvent centrés sur le neuf et sur un inventaire fermé
E-biessa	Domaine spécialisé, recherche hybride, confiance intégrée	Dépend de la qualité des données saisies et validées

1.3 Problématique

Question directrice
Comment offrir une recherche naturelle et rapide de pièces d'occasion, tout en maintenant une compatibilité vérifiable, une identité fiable des utilisateurs et des outils de modération adaptés ?

Trois difficultés structurent le projet. Premièrement, les données véhicule doivent être propres et reliées à chaque annonce sans mélanger des caractéristiques provenant de véhicules différents. Deuxièmement, les rôles doivent limiter chaque action : un acheteur ne publie pas avant validation comme vendeur, un vendeur ne modère pas, et un administrateur dispose d'un historique d'actions. Troisièmement, l'intelligence artificielle doit améliorer la compréhension sans remplacer les règles métier.

1.4 Solution proposée

- une application Angular responsive, utilisable en thème clair ou sombre sans modification de géométrie ;
- un noyau Spring Boot sécurisé pour les comptes, annonces, images, messages, avis et opérations administratives ;
- un service FastAPI pour l'indexation, la recherche vectorielle et l'extraction de contraintes ;
- une base MySQL comme source de vérité et un index Qdrant reconstruit à partir des annonces actives ;
- un modèle Qwen adapté au vocabulaire français, anglais et tunisien, encadré par des validations déterministes ;
- un moteur de recommandations explicable fondé sur les préférences véhicule et les interactions privées ;
- une couche agentique en lecture seule pour orchestrer recherche, compatibilité et recommandation.

1.5 Démarche de réalisation

La réalisation a suivi une démarche incrémentale. Chaque fonctionnalité a été développée, vérifiée et documentée avant l'introduction des fonctions dépendantes. Le socle marketplace a ainsi précédé la recherche sémantique, la personnalisation et les aides multimodales. Cette organisation ne suppose pas la tenue de cérémonies Scrum formelles.

Incrément	Contenu principal	Critère de sortie
Socle	Authentification, rôles, profils, catalogue et annonces	Parcours acheteur/vendeur cohérents
Confiance	Validation vendeur, avis, signalements, administration	Actions auditées et autorisées
Communication	Contact protégé, messagerie et images	Échange temps réel entre deux comptes
IA	Embeddings, Qdrant, Qwen adapté et recherche ancrée	Résultats compatibles et explicables
Personnalisation	Préférences, historique minimal et agents	Suggestions privées, contrôlables et justifiées
Validation	Tests unitaires, intégration et E2E	Scénarios critiques reproductibles

2. Analyse et spécification des besoins

L'analyse transforme la problématique en fonctions mesurables et en contraintes de qualité applicables aux trois profils de la plateforme.

2.1 Identification des acteurs

Acteur	Responsabilités et objectifs
Visiteur	Consulter le catalogue public, filtrer et ouvrir une annonce sans accéder aux coordonnées privées.
Acheteur	Gérer son profil et ses véhicules, sauvegarder, contacter, échanger, signaler et évaluer un vendeur éligible.
Vendeur	Gérer ses annonces, images, compatibilités et statuts, répondre aux conversations et suivre sa réputation.
Administrateur	Valider les vendeurs, modérer annonces et signalements, suspendre des comptes et consulter l'audit.

2.2 Besoins fonctionnels

- inscription par courriel et mot de passe, connexion OAuth facultative, récupération par courriel ou téléphone ;
- édition d'un profil complet avec coordonnées, localisation, activité, biographie et site web ;
- demande de passage vendeur placée en attente jusqu'à décision de l'administrateur ;
- cycle de vie d’une annonce : saisie et aperçu avant publication, modification, activation, désactivation, vente et suppression ; les états persistés sont ACTIVE, INACTIVE et SOLD ;
- ajout, compression, prévisualisation, ordre par glisser-déposer, choix de l'image principale et suppression ;
- recherche par catégorie, marque, modèle, année, carburant, moteur, position, prix, livraison, état et tri ;
- recherche en langage naturel avec contraintes comprises et raisons de correspondance ;
- contact protégé par téléphone, WhatsApp, courriel et messagerie interne avec pièces jointes ;
- avis de 1 à 5 étoiles, commentaire facultatif, réputation agrégée et signalement vendeur ;
- tableau de bord administrateur, files de traitement et journal d'audit ;
- recommandations personnalisées, paramètres de confidentialité et effacement de l'historique de recommandation.

2.3 Besoins non fonctionnels

Axe	Exigence retenue
Sécurité	JWT signé, mots de passe hachés, RBAC, secrets hors dépôt, validation de fichiers et limitation de débit.
Fiabilité	MySQL reste la source de vérité ; un index obsolète ne doit pas exposer une annonce devenue inactive.
Explicabilité	Toute recommandation et tout résultat IA affichent des raisons issues de données contrôlées.
Performance	Pagination, cache court, seuil de pertinence, concurrence bornée et images compressées.
Ergonomie	Interface responsive, états de chargement, erreurs lisibles, thème clair/sombre géométriquement stable.
Maintenabilité	Séparation contrôleur/service/dépôt, DTO dédiés, configuration externalisée et tests automatisés.
Confidentialité	Coordonnées réservées aux acheteurs connectés ; historique IA minimal et effaçable.
Portabilité	Services exécutables localement sur Windows pour la démonstration académique.

2.4 Règles métier centrales

1. Une annonce publique doit être active, appartenir à un vendeur non suspendu et contenir des données valides.

2. Une annonce marquée vendue disparaît immédiatement des listes, recherches et recommandations publiques.

3. Les coordonnées du vendeur ne sont révélées qu'à un acheteur authentifié.

4. Le passage au rôle vendeur exige une demande et une décision explicite de l'administrateur.

5. Une compatibilité multi-véhicule est évaluée ligne par ligne ; le système ne combine jamais la marque d'une ligne avec le carburant d'une autre.

6. La sortie du modèle de langage est une proposition de contraintes, jamais une preuve de compatibilité.

7. Les messages sont réservés aux participants autorisés ; les avis publiés sont consultables sur le profil vendeur.

8. Chaque décision administrative sensible produit une entrée d'audit.

2.5 Matrice de traçabilité

Besoin	Composant responsable	Validation
Authentifier	Spring Security, JWT, OAuth2	Tests de rôles et parcours navigateur
Publier une annonce	PartService, Angular Seller Studio	CRUD, DTO, upload et statuts
Vérifier la compatibilité	FastAPI + règles déterministes Spring	Jeux multilingues et contradictions
Communiquer	MessagingService, WebSocket	Conversation privée et pièces jointes
Modérer	AdminService, audit	Files de signalement et suspension
Recommander	RecommendationService, agents en lecture	Scénarios de préférences, exclusion et diversité

2.6 Diagrammes de cas d’utilisation

La frontière E-biessa comprend Angular, Spring Boot et le service FastAPI. Les acteurs humains déclenchent les opérations autorisées ; les services internes ne sont donc pas des acteurs externes dans ces diagrammes. Les vues regroupent les opérations proches pour conserver une lecture globale. Une association indique une interaction, pas une relation de succession.

Parcours de consultation et d’achat

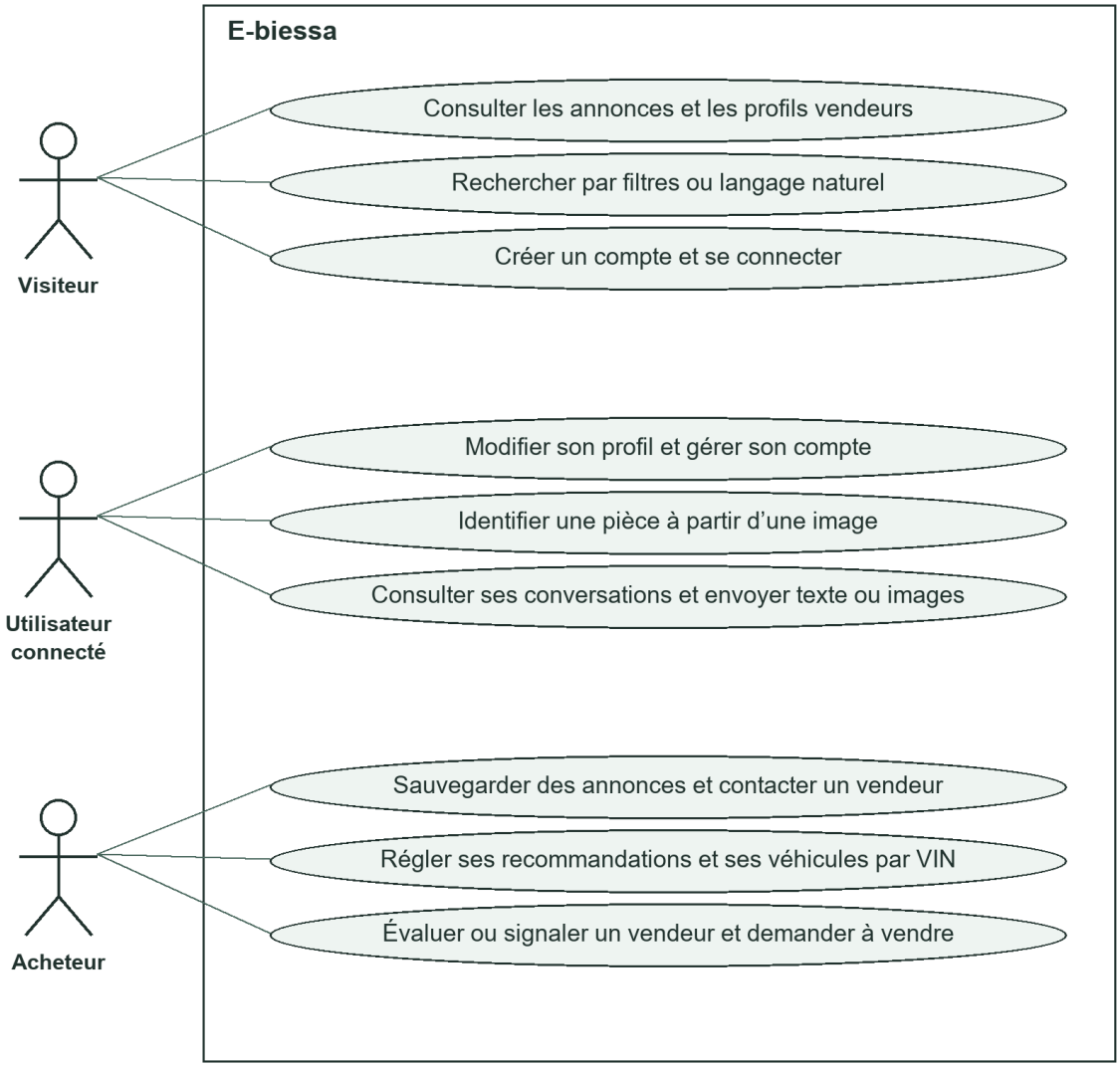


Figure 1 - Cas d’utilisation de consultation et d’achat

Les fonctions d’utilisateur connecté sont communes aux rôles authentifiés. Les coordonnées, les avis et les recommandations personnalisées restent soumis aux contrôles du rôle acheteur. L’expression « achat » désigne ici la recherche et la mise en relation, sans paiement intégré.

Publication et administration

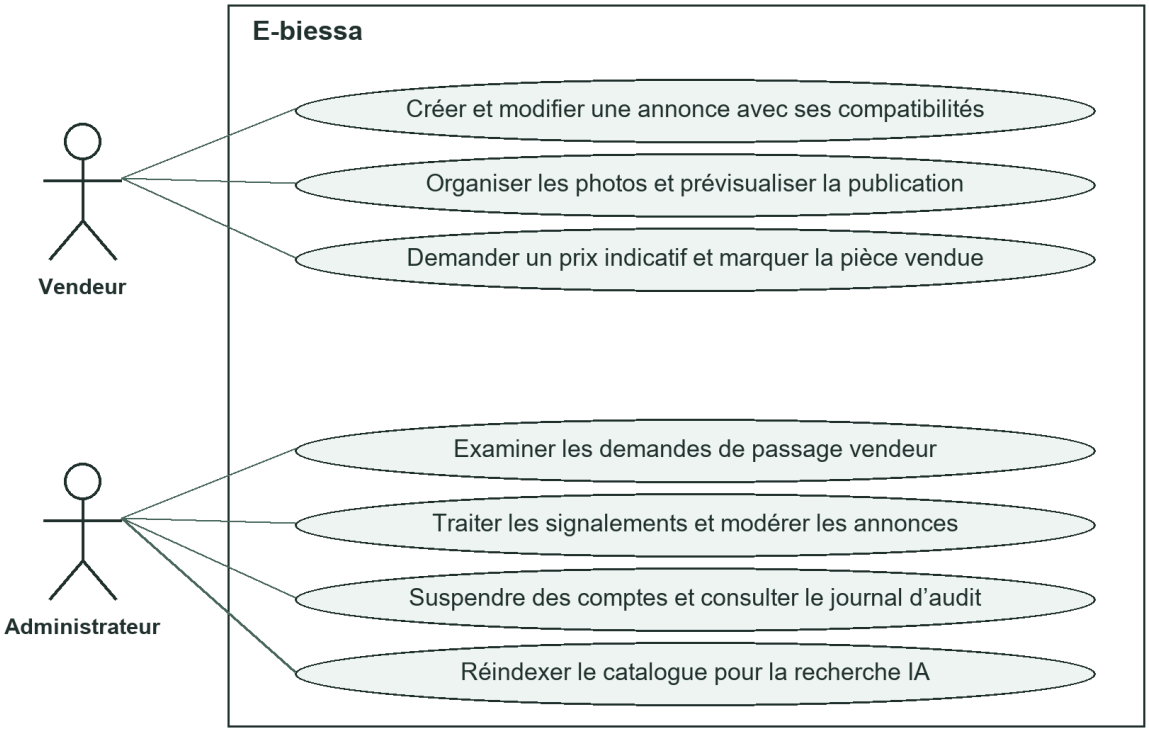


Figure 2 - Cas d'utilisation de vente et d'administration

Le vendeur bénéficie des fonctions communes de compte et de messagerie. L'administrateur possède également les droits de gestion des annonces autorisés par le backend. La suspension et la résolution des signalements donnent lieu à des traces d'audit.

Services externes et variantes

Service externe	Interaction avec E-biessa
Google, Facebook, Microsoft	Connexion OAuth quand le fournisseur est configuré.
SMTP et Twilio	Acheminement des codes de récupération selon la configuration.
CarQuery et NHTSA vPIC	Catalogue automobile et décodage VIN, avec limites de couverture.
Cloudinary	Stockage optionnel des images à la place du stockage local.
Service vocal du navigateur	Transcription soumise au support et aux permissions du navigateur.

Pour publier, le vendeur saisit les informations, consulte l'aperçu puis confirme l'envoi ; le serveur valide les champs et la propriété. Pour devenir vendeur, l'acheteur transmet sa demande et reste acheteur pendant l'examen. L'administrateur accepte ou refuse ; le rôle ne change qu'en cas d'acceptation. Pour contacter un vendeur, l'acheteur se connecte, ouvre une annonce active puis utilise les coordonnées autorisées ou crée une conversation.

3. Conception

La conception sépare les responsabilités transactionnelles, l'expérience utilisateur et le calcul d'intelligence artificielle afin de réduire le couplage et de garder les décisions critiques vérifiables.

3.1 Choix d'architecture

Le cahier des charges initial mettait l'accent sur FastAPI. Or, le socle transactionnel du projet a été stabilisé en Spring Boot. La décision finale conserve Spring comme coeur métier et exploite FastAPI comme microservice IA. Cette adaptation respecte les capacités demandées tout en évitant une réécriture risquée.

Décision structurante
Spring Boot est la frontière de sécurité et la source des réponses publiques. FastAPI ne reçoit que les documents nécessaires à l'indexation et renvoie des identifiants classés. Avant affichage, Spring recharge toujours les annonces depuis MySQL.

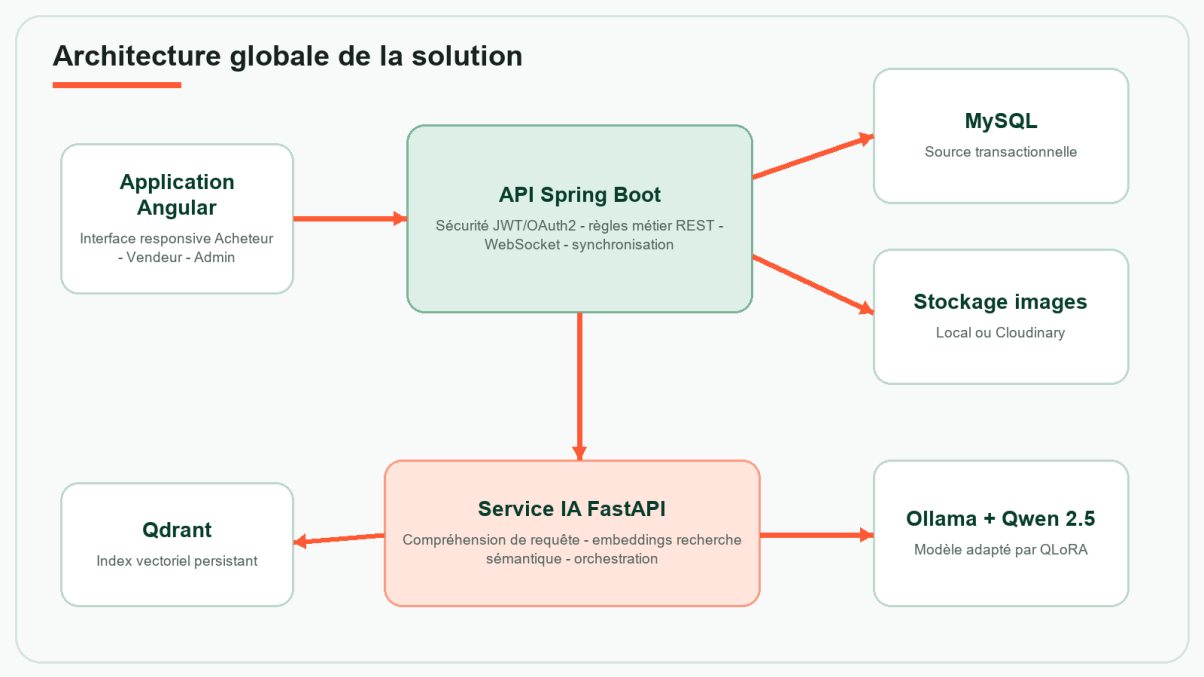


Figure 3 - Architecture globale de la solution

3.2 Architecture logique

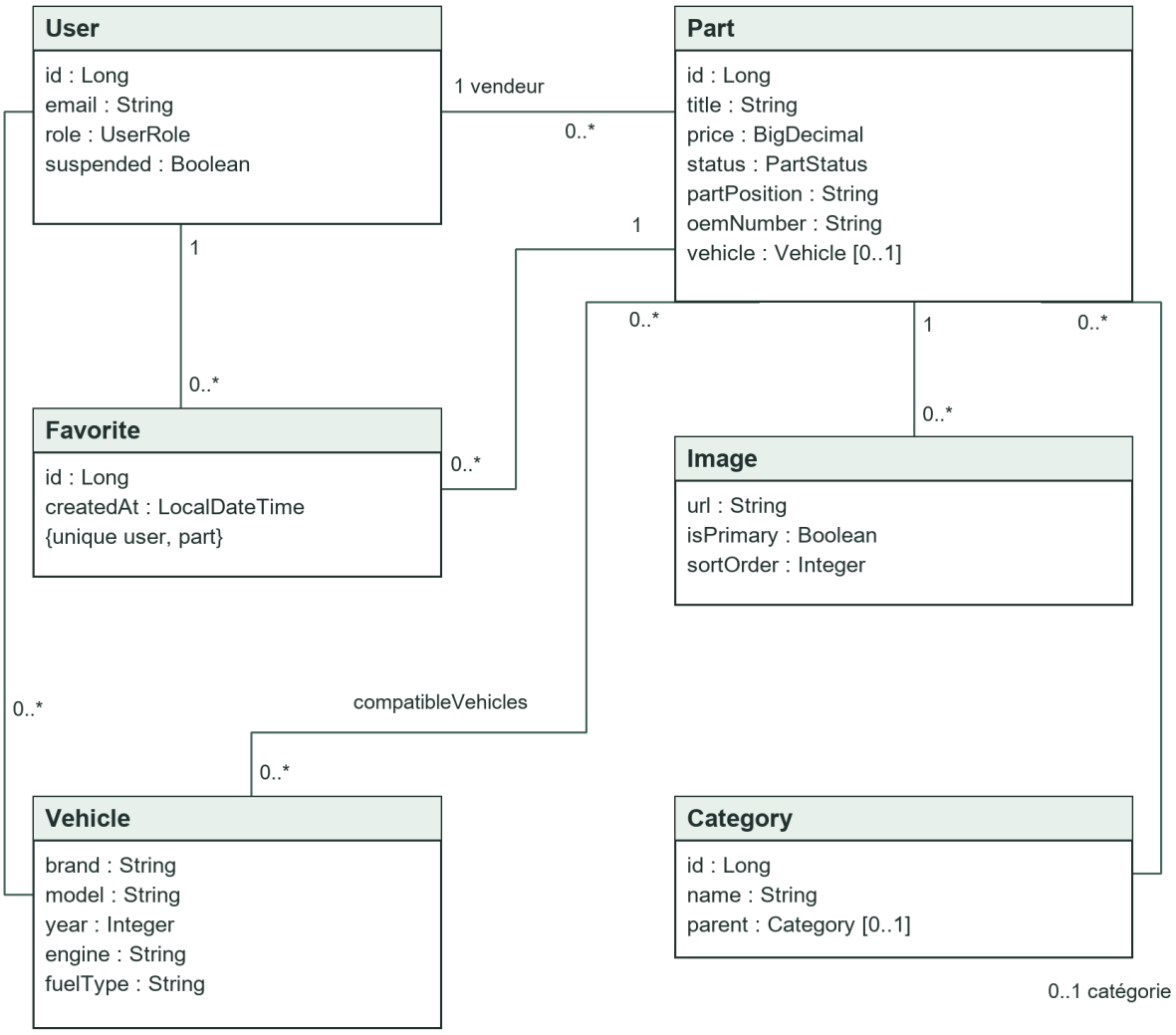
Couche	Responsabilité
Présentation	Composants Angular, routes, formulaires, thèmes, retours d'erreur et accessibilité.
API	Contrôleurs REST, validation des DTO, sérialisation stable et contrats HTTP.
Métier	Services Spring portant autorisation, règles de cycle de vie, modération et classement.
Persistance	Dépôts JPA et modèle relationnel MySQL ; Qdrant pour les vecteurs reconstruisibles.
IA	Embeddings, extraction de contraintes, récupération, agents spécialisés et explications.
Infrastructure	Stockage d'images, courriel/SMS optionnels, OAuth et catalogue véhicule externe avec repli local.

3.3 Modèle de données

L'entité Utilisateur centralise l'identité et le rôle. Une annonce appartient à un vendeur, à une catégorie et à une collection de véhicules compatibles. Les images conservent un ordre et une image principale. Les conversations relient un acheteur, un vendeur et l'annonce d'origine. Les signaux d'activité restent séparés des messages et des coordonnées afin de limiter les données utilisées pour la personnalisation.

Les quatre vues suivantes forment le diagramme de classes du domaine persistant. Les noms correspondent aux entités Java ; les attributs sont sélectionnés pour la lisibilité. Les associations portent leurs multiplicités. Les rôles BUYER, SELLER et ADMIN sont les valeurs de UserRole, et non des sous-classes de User. Les méthodes métier appartiennent aux services, non aux entités représentées.

Catalogue et favoris

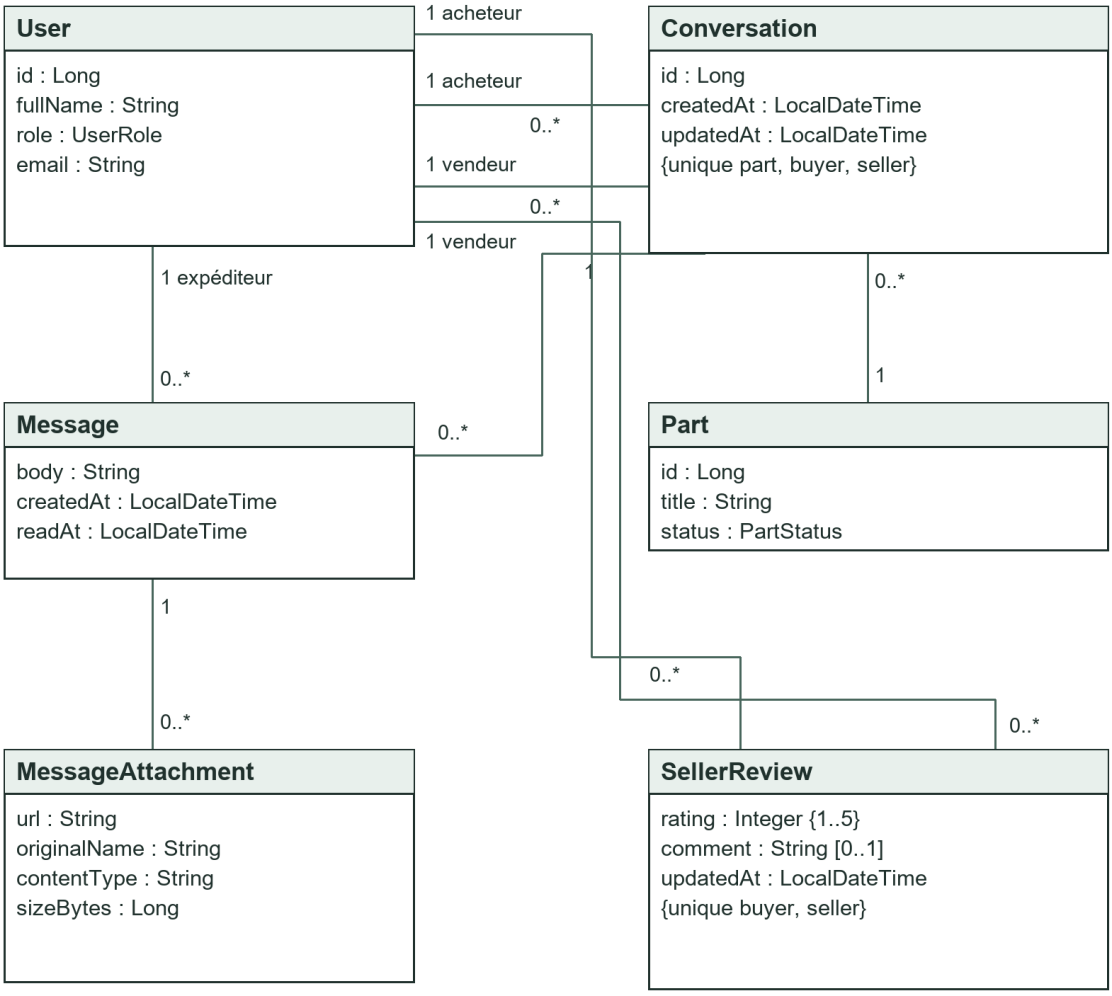


Préférences User ↔ Vehicle : 0..* ↔ 0..* en base ; au plus 5 côté acheteur.

Figure 4 - Classes du catalogue et des favoris

Part et Vehicle sont reliés par part_compatible_vehicles. Le lien historique Part.vehicle est maintenu séparément pour la compatibilité des données existantes. Category possède un parent facultatif et zéro à plusieurs sous-catégories. Les restrictions moteur et carburant de Part complètent les caractéristiques des véhicules ; elles ne constituent pas une entité de compatibilité distincte.

Conversations et réputation

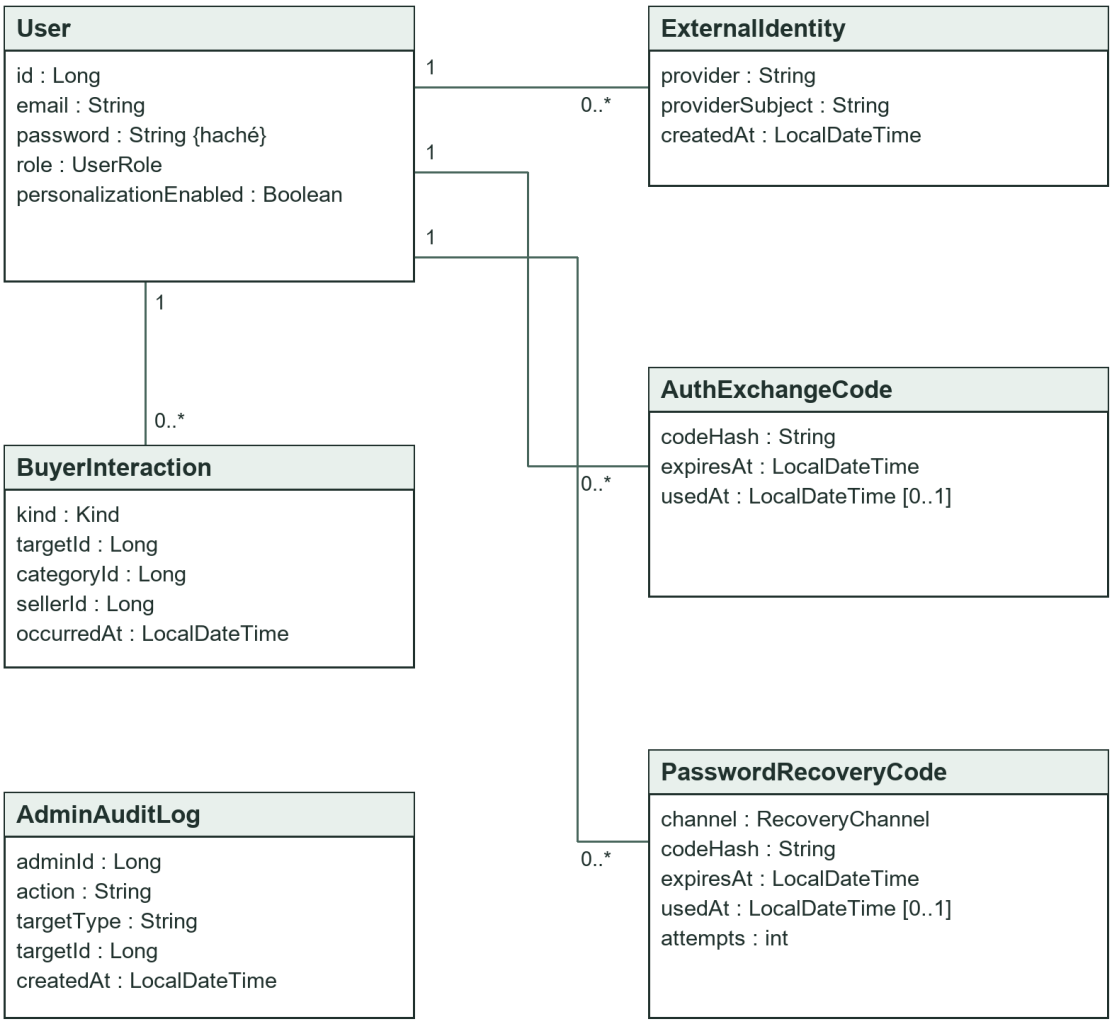


Chaque avis concerne un acheteur et un vendeur ; la conversation établit l'éligibilité métier, sans relation JPA directe entre avis et conversation.

Figure 5 - Classes de communication et de réputation

Une conversation relie exactement une annonce, un acheteur et un vendeur. Elle contient zéro à plusieurs messages ; chaque message a un seul expéditeur et zéro à plusieurs pièces jointes. Les avis référencent directement les deux utilisateurs. Les autorisations vérifient que l'expéditeur appartient à la conversation.

Identité et personnalisation

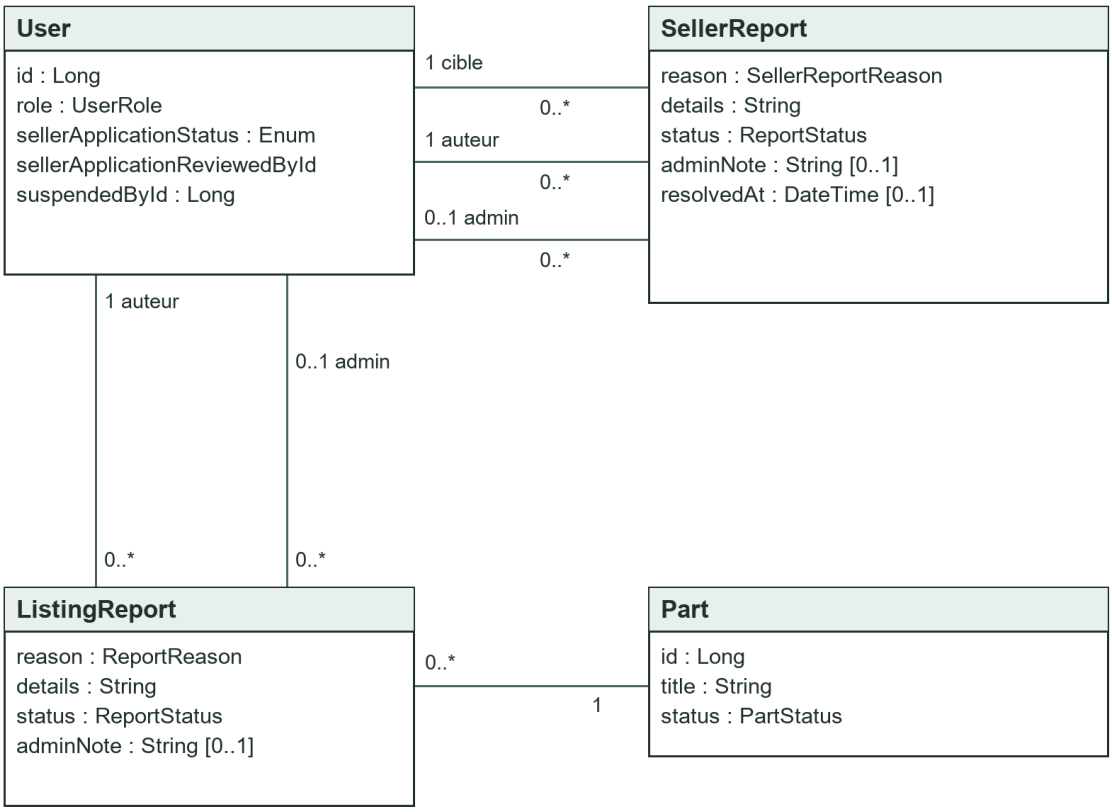


AdminAuditLog conserve des valeurs de référence, pas une association JPA.

Figure 6 - Classes d'identité et de personnalisation

Les codes temporaires sont associés à un utilisateur et stockés sous forme de condensat. BuyerInteraction contient des références numériques de contexte et n'établit pas de relations JPA vers Part ou Category. AdminAuditLog conserve un instantané d'identifiants et de libellés pour préserver la trace d'une action.

Signalements et validation des vendeurs



La demande vendeur est portée par User ; aucune classe SellerApplication séparée.

Figure 7 - Classes de modération

SellerReport et ListingReport distinguent l'auteur, la cible et l'administrateur résolveur facultatif. La demande de passage vendeur est représentée par des champs de User, dont le statut et les dates de traitement. L'acceptation change le rôle ; le dépôt d'une demande ne crée pas automatiquement un vendeur.

Les structures de recherche, VIN, prix et vision sont des DTO ou des résultats temporaires de services. Elles ne créent pas de tables JPA supplémentaires. L'index Qdrant est reconstructible à partir de MySQL et n'est pas un second référentiel transactionnel. Les champs d'identité sont réservés aux DTO autorisés et ne sont pas tous exposés dans les profils publics.

3.4 Conception de la recherche intelligente

Une annonce active est transformée en document textuel normalisé contenant titre, description, catégorie, références et compatibilités. FastEmbed calcule son vecteur multilingue, stocké dans la collection Qdrant. Lors d'une requête, Qwen extrait un objet JSON. Le système fusionne cette sortie avec les contraintes lexicales et les filtres explicites, puis récupère des candidats sémantiques.

Le classement final applique un seuil minimal et vérifie les contraintes sur une même ligne de compatibilité. Les raisons visibles - par exemple « compatible Golf », « diesel », « position arrière » ou « système de freinage » - proviennent des champs réellement associés à l'annonce. La réponse est construite par Spring à partir des DTO actifs, ce qui empêche une prose générée ou un identifiant périmé de devenir une recommandation publique.

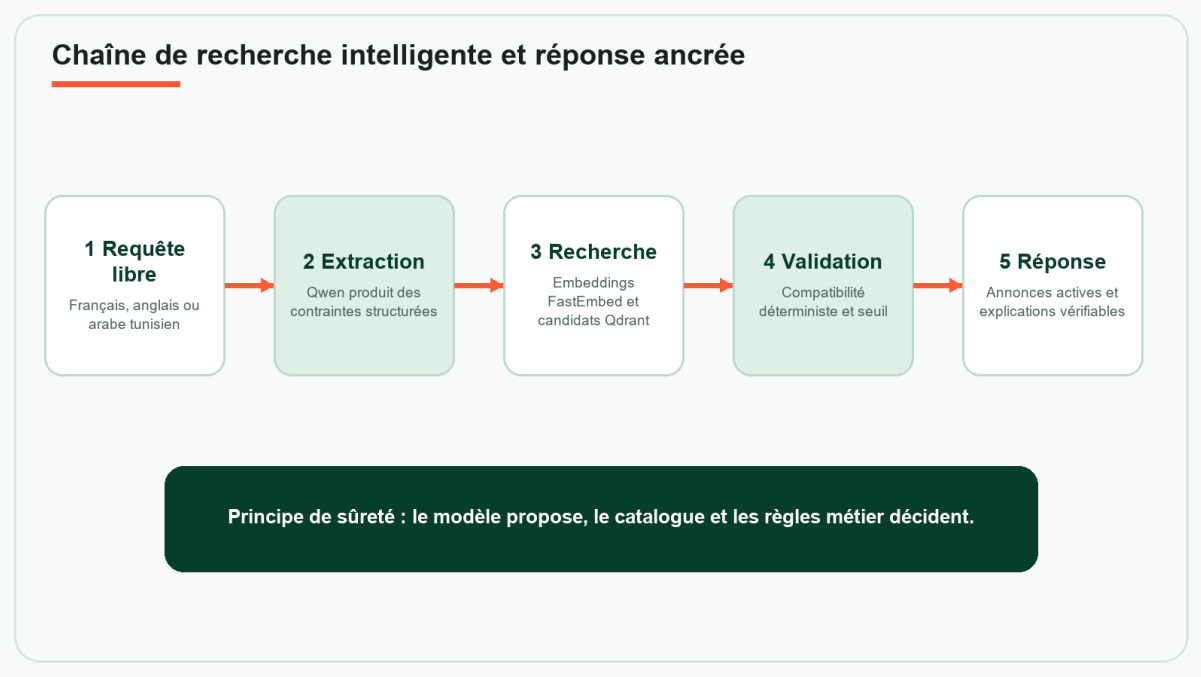


Figure 8 - Chaîne de recherche intelligente et réponse ancrée

3.5 Orchestration agentique finale

La couche LangChain encapsule trois outils StructuredTool : search_agent, compatibility_agent et recommendation_agent. Le routage est déterministe : la recherche s'exécute d'abord, la vérification des contraintes suit si nécessaire, puis les candidats personnalisés fournis par Spring peuvent être intégrés selon l'intention détectée. Il ne s'agit pas d'agents autonomes qui planifient librement des actions. Aucun outil ne crée, ne modifie ou ne supprime une donnée métier.

Agent	Entrées	Sortie autorisée	Garde-fou
Recherche	Texte et filtres	Identifiants candidats	Limite et seuil de pertinence
Compatibilité	Contraintes et fiche annonce	Candidats conservés après revalidation	Règles catalogue prioritaires

Agent	Entrées	Sortie autorisée	Garde-fou
Recommandation	Candidats assainis fournis par Spring et limite	Suggestions et raisons	Pas de données inter-utilisateurs
Orchestrateur	Intention utilisateur	Plan court et réponse structurée	Lecture seule, délai et nombre d'appels bornés

3.6 Sécurité

- identité extraite du contexte JWT, jamais d'un identifiant fourni librement par le client ;
- autorisation par rôle au niveau des routes et réévaluation de la propriété dans les services ;
- codes OAuth courts, à usage unique, sans JWT dans l'URL de redirection ;
- codes de récupération expirables, nombre d'essais limité et réponses non révélatrices ;
- jetons WebSocket courts et à usage unique pour éviter de transmettre le JWT dans une chaîne de requête ;
- signature réelle des fichiers image contrôlée en plus du type MIME déclaré ;
- jeton interne partagé entre Spring et FastAPI pour les opérations d'indexation ;
- limitation de débit, concurrence bornée, cache court et délai maximum sur l'endpoint IA public ;
- secrets chargés depuis l'environnement et exclus de Git.

3.7 Conception de la messagerie

La messagerie suit un modèle conversationnel unique par contexte d'annonce. Le chargement initial place la vue sur le dernier message, tandis que le contenu reste dans une zone de hauteur limitée et défilable. Les événements WebSocket mettent à jour l'interface sans recharger la page. Les images jointes sont stockées via le même contrat que les images d'annonces, avec des limites dédiées.

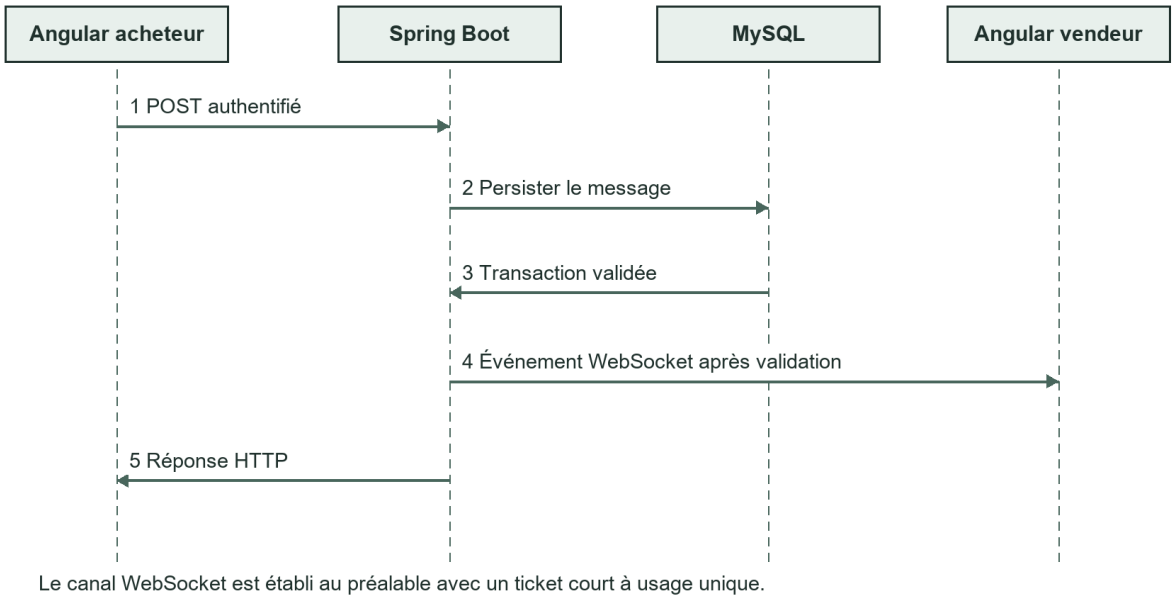


Figure 9 - Séquence simplifiée de la messagerie instantanée

3.8 Conception de la recommandation

Le moteur combine compatibilité déclarée, catégories sauvegardées, vendeurs contactés, popularité et récence. Les préférences véhicule agissent comme contrainte forte. Les interactions comportementales sont dédupliquées, limitées à 200 enregistrements, pondérées avec une demi-vie de trente jours et ignorées après quatre-vingt-dix jours. Le score reste un rang interne et n'est jamais présenté comme un pourcentage de compatibilité.

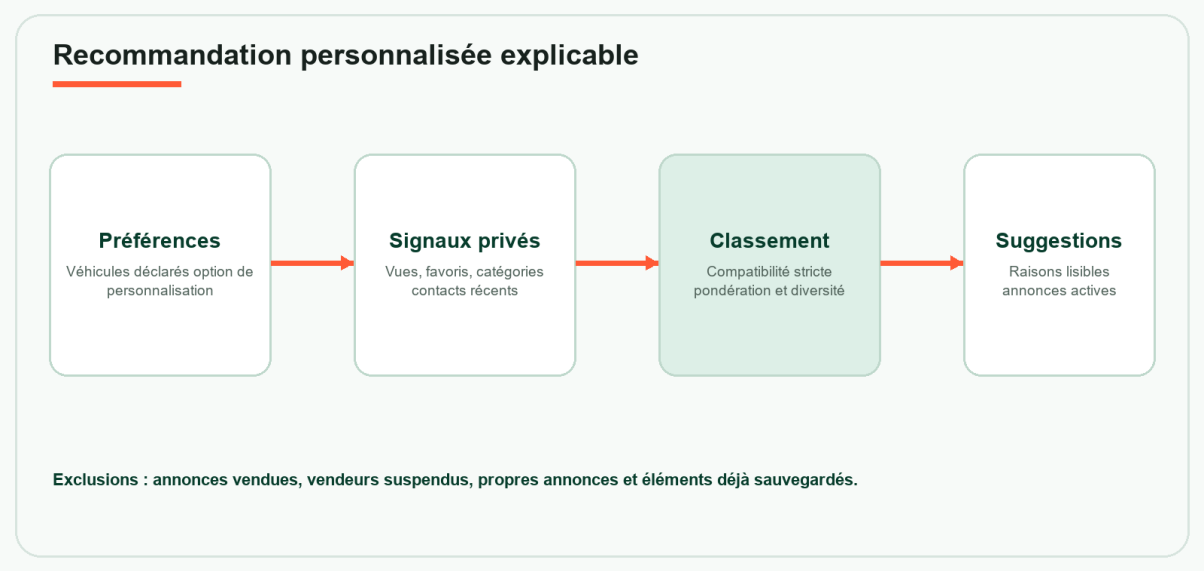


Figure 10 - Recommandation personnalisée explicable

4. Réalisation

Cette partie présente l'environnement technique, les principaux modules développés et l'intégration finale des services.

4.1 Environnement de travail

Élément	Choix final	Rôle
Langage backend	Java 21	API transactionnelle et règles métier
Framework backend	Spring Boot 3.2.8	Web, sécurité, validation, JPA et WebSocket
Frontend	Angular 21, TypeScript 5.9	SPA responsive et formulaires réactifs
Service IA	Python 3.12, FastAPI	Recherche sémantique et orchestration
Données	MySQL, Qdrant	Source de vérité relationnelle et index vectoriel
Modèle	Qwen 2.5 7B Instruct	Extraction multilingue de contraintes
Adaptation	Unsloth QLoRA	Entraînement local quantifié
Exécution LLM	Ollama	Inférence locale sur GPU
Tests	JUnit, Vitest, Pytest, Playwright	Validation multi-couche
Versionnement	Git et GitHub	Historique, partage et sauvegarde du code

4.2 Backend Spring Boot

Le backend applique une organisation par contrôleurs, services, dépôts, entités et DTO. Les contrôleurs exposent les contrats HTTP et délèguent les décisions aux services. Les DTO empêchent la sérialisation directe des entités JPA et stabilisent les données visibles. Le gestionnaire global d'exceptions transforme les erreurs en messages exploitables par Angular.

Les domaines couverts sont l'authentification, les utilisateurs, les catégories, les véhicules, les annonces, les images, les favoris, les conversations, les avis, les signalements, les recommandations et l'administration. Les événements de synchronisation avec l'IA sont publiés uniquement après la validation de la transaction ; une panne de l'index n'annule donc pas une opération légitime sur le catalogue.

4.3 Frontend Angular

L'application Angular est organisée en vues autonomes : marketplace, détail d'annonce, favoris, profil, profil public du vendeur, studio vendeur, boîte de réception et tableau de bord administrateur. La navigation et les actions changent selon le rôle. Les formulaires signalent les champs invalides et affichent les messages d'erreur retournés par l'API.

Le thème clair utilise des surfaces légèrement teintées et des accents verts et orange pour délimiter les zones, tandis que le thème sombre conserve exactement les mêmes espacements, bordures et tailles. La bascule de thème modifie seulement les variables de couleur, ce qui supprime les glissements de mise en page.

Principe d'interface
Une action métier importante possède toujours un état visible : chargement, succès, erreur récupérable ou confirmation. Les listes sont paginées et les filtres restent dans l'URL afin de pouvoir revenir à la même recherche.

4.4 Gestion complète des annonces

Bloc	Informations prises en charge
Identification	Titre, description, catégorie, fabricant, marque de pièce, OEM principal et alternatives
État commercial	Prix, négociable, état, garantie, livraison, collecte et localisation
Compatibilité	Plusieurs véhicules, année, moteur, carburant et position
Médias	JPEG/PNG/WebP, compression, ordre, image principale et suppression
Cycle de vie	Aperçu avant envoi ; états ACTIVE, INACTIVE, SOLD ; suppression séparée
Découverte	Filtres, tri, pagination, favoris, recherche IA et recommandations

4.5 Comptes, profils et confiance

L'identité locale est liée à l'adresse électronique et peut être complétée par Google, Facebook ou Microsoft lorsque les identifiants OAuth correspondants sont configurés. Le profil séparé contient nom, téléphone, localisation, nom commercial, site web et biographie. L'acheteur qui souhaite vendre soumet ces informations ; son statut reste en attente jusqu'à la décision d'un administrateur.

Un acheteur ayant déjà ouvert une conversation avec un vendeur peut lui attribuer de une à cinq étoiles et ajouter un commentaire facultatif. Un seul avis est conservé par couple acheteur–vendeur et peut être mis à jour. Cette condition atteste un contact, pas un achat confirmé : aucune commande payée n’est enregistrée dans la plateforme. La moyenne et le nombre d’avis sont publics. Les signalements et leur résolution administrative sont gérés séparément.

4.6 Contact et messagerie

La fiche annonce comporte une section « Contacter le vendeur ». Pour réduire l'extraction automatisée, les coordonnées ne sont renvoyées qu'à un acheteur connecté. Les boutons génèrent un appel téléphonique, un message WhatsApp préparé avec le titre de la pièce ou un courriel. L'acheteur peut aussi démarrer une conversation interne.

La boîte de réception présente une liste organisée de conversations et un panneau de discussion. Le fil possède une hauteur limitée, s'ouvre sur le dernier message et conserve le défilement vers les anciens échanges. L'envoi d'une image permet au vendeur de fournir des vues supplémentaires d'une pièce sans exposer un canal externe.

4.7 Mise en oeuvre de l'IA

Le modèle de base Qwen 2.5 7B Instruct a été chargé en 4 bits puis adapté avec Unsloth et LoRA de rang 8 sur un ordinateur équipé d'une RTX 4060 de 8 Go de VRAM et de 16 Go de mémoire système. Le jeu d'apprentissage contient des formulations anglaises, françaises et tunisiennes ainsi que les sorties JSON canoniques attendues. Une passe d'entraînement a été retenue après un test court, car les erreurs restantes concernaient surtout la copie de chiffres et relevaient mieux d'une validation déterministe que d'époques supplémentaires.

Le corpus corrigé v2 comprend 1 120 exemples d’apprentissage, 160 de validation et 120 de test. Les fichiers d’évaluation sont conservés séparément. La configuration locale limite la séquence à 256 tokens, utilise un lot de taille 1 et accumule les gradients sur huit étapes. Le taux d’apprentissage retenu pour l’époque v2 est de 0,00005.

Le modèle exporté est servi localement par Ollama sous le nom e-biessa-query-v2. FastAPI lui demande uniquement un objet structuré. La fonction de fusion contrôle ensuite l'année explicite, normalise les catégories et vérifie chaque valeur dans le catalogue. Cette approche apporte la souplesse linguistique du LLM tout en gardant un comportement reproductible.

4.8 Jeu de démonstration

Le profil de développement initialise un administrateur, trois acheteurs, trois vendeurs et soixante annonces réparties entre plusieurs marques, modèles, catégories et villes. Chaque vendeur possède plusieurs annonces détaillées. Les données sont réutilisables sans duplication au redémarrage et servent uniquement à la démonstration académique.

Donnée	Volume de démonstration
Administrateurs	1
Acheteurs	3
Vendeurs	3
Annonces actives initiales	60
Définitions véhicule locales	20
Langues d'exemple IA	anglais, français, arabe tunisien translittéré

4.9 Configuration et exécution locale

1. Démarrer MySQL et créer la base marketplace.

2. Définir les variables DB_URL, DB_USERNAME, DB_PASSWORD, JWT_SECRET et CORS_ALLOWED_ORIGINS dans l'environnement local de Spring.

3. Définir le même AI_SERVICE_TOKEN dans Spring et FastAPI, puis activer l'intégration IA.

4. Lancer Ollama avec le modèle adapté e-biessa-query-v2.

5. Démarrer FastAPI sur 127.0.0.1:8001, Spring Boot sur localhost:8080 et Angular sur localhost:4200.

6. Exécuter une réindexation complète avec le compte administrateur avant la première recherche sémantique.

4.10 Assistance VIN et catalogue automobile

Le profil acheteur propose la saisie d'un VIN de 17 caractères. Spring contrôle le format, interroge NHTSA vPIC puis rapproche la marque, le modèle, l'année et le carburant des lignes Vehicle du catalogue local. L'interface propose uniquement les identifiants locaux reconnus ; l'acheteur doit confirmer l'enregistrement de ses préférences. Une absence de correspondance conduit à la sélection manuelle, sans création automatique d'une compatibilité.

Le décodage utilise un délai configuré et un cache temporaire, fixé par défaut à 24 heures. Le VIN n'est pas persisté en base ; il est néanmoins transmis à NHTSA et intervient dans le cache de requête. La couverture vPIC, principalement orientée vers le marché américain, limite la reconnaissance de certains véhicules européens ou tunisiens. Le décodage n'est jamais présenté comme une garantie de montage.

4.11 Estimation indicative du prix

PriceEstimatorService compare la saisie du vendeur à un maximum de vingt annonces actives. Le score tient compte de la référence OEM, de la catégorie, des véhicules partagés, des termes du titre, de la position, de la marque de pièce et de l'état. L'annonce en cours d'édition et les vendeurs suspendus sont exclus. En l'absence de comparables suffisants, le système ne fabrique pas de prix.

La réponse fournit un prix central, une plage interquartile, le nombre de comparables et un niveau de confiance heuristique. Les quantiles de l'implémentation correspondent au rang arrondi dans la liste triée ; pour un effectif pair, le prix central n'est donc pas nécessairement la moyenne des deux valeurs centrales. Le calcul porte sur des prix demandés, pas sur des ventes conclues. Le vendeur conserve la décision et doit explicitement appliquer la suggestion.

4.12 Identification de pièces par image

L'utilisateur connecté peut transmettre une image JPEG, PNG ou WebP, limitée à 5 Mo. Spring applique l'authentification, la limitation de débit et les validations avant de transmettre les octets au service FastAPI avec le jeton interne. Le module optionnel utilise openai/clip-vit-base-patch32 pour comparer l'image à une taxonomie de vingt-deux types de pièces et retourner au plus cinq candidats.

Il s'agit d'une classification zero-shot, distincte de l'adaptation QLoRA de Qwen. Les scores sont relatifs aux catégories proposées, et non des probabilités calibrées de justesse. Le système ne déduit ni véhicule exact, ni référence OEM, ni garantie de compatibilité à partir des pixels. Le candidat choisi alimente une recherche dont les règles déterministes restent actives.

VISION_ENABLED reste désactivé par défaut. L’activation requiert les dépendances de vision et le téléchargement initial du modèle, mis en cache pour les requêtes suivantes. Aucune fonction métier ne conserve la photo d’identification ni ne l’utilise pour l’apprentissage ; le traitement multipart peut toutefois recourir à des fichiers temporaires selon la configuration du serveur. Les tests automatisés isolent l’inférence au moyen d’un substitut.

4.13 Saisie vocale

Angular exploite la reconnaissance vocale proposée par le navigateur avec un choix français de Tunisie, arabe de Tunisie ou anglais américain. Le texte reconnu reste modifiable et la recherche ne part qu’après confirmation. L’absence de microphone, le refus de permission ou l’absence de prise en charge laisse disponible la saisie au clavier.

E-biessa reçoit la transcription, pas l’enregistrement audio. Le navigateur peut cependant faire appel au service vocal de son fournisseur ; la fonctionnalité ne doit pas être décrite comme une reconnaissance entièrement locale. Les tests utilisent un objet de reconnaissance simulé ; la qualité acoustique dépend d’une vérification réelle sur le navigateur et le microphone retenus.

5. Tests et validation

La validation combine des tests rapides par composant et des parcours représentatifs. Les chiffres présentés correspondent aux campagnes automatisées conservées avec le projet et à des jeux de données contrôlés.

5.1 Stratégie

La pyramide de tests couvre d'abord les règles métier isolées, puis les intégrations avec une base de test, les contrats du service IA et enfin les parcours dans un navigateur réel. Les scénarios sensibles - identité, propriété d'une annonce, compatibilité et visibilité des éléments vendus - sont testés à plusieurs niveaux.

Niveau	Outil	Portée
Backend	JUnit, Spring Boot Test, H2	Services, contrôleurs, sécurité, sérialisation et persistance
Frontend	Vitest	Composants, formulaires, thèmes, erreurs et recommandations
IA	Pytest	Sécurité interne, normalisation, recherche et garde-fous
Navigateur	Playwright	Rôles, recherche IA, thèmes et parcours principaux
Modèle	Jeu de validation tenu à l'écart	JSON et exactitude par champ

5.2 Résultats automatisés

Suite	Résultat validé	Observation
Spring Boot	62 tests réussis	Rapports JUnit locaux : sécurité, recommandations, VIN et prix
Angular	51 tests réussis	Dernière campagne documentée ; image et voix incluses
FastAPI	28 tests réussis	Dernière campagne documentée ; CLIP simulé dans les tests
Playwright isolé	7 parcours réussis	Navigateur réel avec API contrôlée
Playwright pile complète	4 scénarios opt-in non exécutés	Nécessite MySQL, Spring, FastAPI, Ollama et Angular actifs

Lecture correcte des résultats
Les tests Playwright isolés prouvent le comportement de l'interface dans un navigateur, mais ne remplacent pas une campagne sur la pile locale complète. Les deux suites sont donc conservées séparément.

Ces nombres décrivent les dernières campagnes consignées avant la revue documentaire, et non une nouvelle exécution de toutes les suites. Les rapports JUnit présents totalisent 62 tests sans échec. Les essais navigateur avec API simulée restent distincts des quatre scénarios de pile complète, désactivés par défaut. Ni ces comptes ni les simulations de CLIP et de microphone ne démontrent la précision de la vision ou de la reconnaissance vocale sur des données réelles.

5.3 Évaluation du modèle de compréhension

L'évaluation conservée dans training/v2-results-120.json porte sur 120 exemples tenus à l'écart de l'apprentissage. Elle mesure séparément la sortie brute de l'adaptateur et la chaîne corrigée. Un JSON syntaxiquement valide ne garantit pas la conformité du type de chaque champ, notamment celui de l'année.

Mesure	Résultat final
JSON valide	99,2 % - 119/120
Marque	99,2 %
Modèle	96,7 %
Carburant	99,2 %
Position	99,2 %
Catégorie canonique	99,2 %

Le type de pièce canonique atteint 99,2 %. En revanche, l'année brute obtient 0 % d'exactitude stricte et l'enregistrement brut complet obtient également 0 %. Après validation autoritaire de l'année explicite, l'exactitude de l'enregistrement complet atteint 96,7 % (116/120). Ce dernier chiffre caractérise donc la chaîne hybride, et non le modèle seul. Le petit jeu synthétique et les formes linguistiques qu'il couvre limitent toute généralisation à des utilisateurs réels.

5.4 Validation de la recherche et des recommandations

Le benchmark conservé dans training/retrieval-results.json comprend six requêtes synthétiques liées au catalogue de démonstration. HitRate@12, le rang réciproque moyen et l'exactitude des contraintes valent 1,0 sur ce jeu. Les six requêtes ne constituent ni une campagne statistique représentative ni une preuve d'efficacité en conditions réelles.

Les recommandations sont vérifiées sur le démarrage à froid, la correspondance exacte multi-véhicule, les carburants contradictoires, la priorité aux catégories sauvegardées, les vendeurs contactés, la diversité, les restrictions de rôle, l'effacement de l'historique et l'exclusion d'annonces supprimées. Les raisons retournées proviennent des mêmes signaux que le score.

5.5 Recette fonctionnelle

Scénario	Résultat attendu
Acheteur recherche un étrier avant pour Golf diesel 2020	Contraintes reconnues, aucune pièce arrière affichée
Vendeur marque une annonce vendue	Masquage immédiat des résultats publics ; synchronisation de l'index après transaction
Acheteur ouvre les coordonnées	Accès accordé uniquement avec le rôle Acheteur actif
Acheteur joint une photo à un message	Fichier validé, stocké et diffusé aux deux participants
Acheteur demande à devenir vendeur	Statut en attente jusqu'à décision administrateur
Administrateur suspend un vendeur	Connexion bloquée, annonces masquées et action auditée
Acheteur désactive la personnalisation	Historique comportemental supprimé, préférences explicites conservées
Index Qdrant contient un ID obsolète	Spring l'écarte lors du rechargement MySQL

5.6 Limites et mesures compensatoires

Limite	Mesure intégrée
Catalogue CarQuery parfois indisponible	Repli local et confirmation explicite de la compatibilité par le vendeur
Petit jeu d'évaluation IA	Résultats présentés comme techniques, sans généralisation abusive
Données de démonstration	Comptes fictifs limités à la démonstration ; renouvellement avant diffusion
LLM sujet aux erreurs	Sortie structurée, normalisation, seuil et règles déterministes
Classement personnalisé content-based	Explications, désactivation, effacement et préparation à des évaluations humaines
Exécution académique locale	Architecture documentée et configuration externe pour une future mise en ligne

5.7 Couverture du cahier des charges

Périmètre	Réalisation et portée
Phase 1 marketplace	Comptes, annonces, filtres, images, contact et administration ; transactions hors plateforme.
Phase 2 intelligence artificielle	Embeddings, Qdrant, extraction Qwen adaptée et réponses déterministes vérifiées.
Phase 3 orchestration	Outils LangChain en lecture seule, routage déterministe et recommandations explicables.
Extensions proposées	VIN, prix indicatif, identification CLIP optionnelle et saisie vocale.
Choix d’architecture	Spring Boot conservé pour le métier ; FastAPI consacré à l’IA, contrairement à un backend entièrement Python.
Limites de validation	Pas d’étude utilisateur représentative ; vision, voix et pile complète à qualifier sur les services réels.
Hors périmètre académique	Déploiement public, paiement, commandes, catalogue OEM certifié et garantie de compatibilité.

Le projet est achevé comme démonstrateur académique intégrant ces fonctions. La disponibilité des fournisseurs OAuth, du courriel, du SMS, de Cloudinary et de CLIP dépend des paramètres et des services activés. Le déploiement public, les migrations de schéma versionnées, la montée en charge et une étude utilisateur demeurent des travaux distincts ; ils ne sont pas présentés comme déjà validés.

Conclusion générale

Le projet E-biessa aboutit à une plateforme complète de mise en relation autour des pièces automobiles d'occasion. Le socle transactionnel prend en charge les comptes, les rôles, les profils, les annonces riches, les images, la compatibilité multi-véhicule, les favoris, le contact, la messagerie, la réputation et l'administration. L'expérience Angular rend ces fonctions accessibles dans une interface cohérente et responsive.

L'apport majeur est l'intégration contrôlée de l'intelligence artificielle. Le modèle adapté comprend des formulations multilingues proches du langage des utilisateurs. La recherche vectorielle améliore le rappel, tandis que la validation déterministe protège la précision. Les recommandations et les agents spécialisés utilisent les mêmes principes : lecture seule, données minimales, raisons visibles et retour systématique à la source de vérité.

Ce travail a permis de mobiliser la conception orientée domaine, la sécurité web, les architectures distribuées, l'UX, le traitement du langage et l'évaluation expérimentale. La séparation Spring Boot / FastAPI représente un compromis justifié entre robustesse métier et agilité IA.

Perspectives

- déployer la pile derrière HTTPS après rotation et gestion centralisée des secrets ;
- faire valider la compatibilité par des professionnels et intégrer des catalogues OEM sous licence ;
- constituer un benchmark plus large, annoté manuellement, couvrant dialectes et fautes d'orthographe ;
- mesurer la qualité des recommandations avec des retours anonymisés et des métriques hors ligne ;
- ajouter notifications, paiement ou réservation uniquement après validation juridique et opérationnelle ;
- surveiller dérive du modèle, qualité des données et taux de signalement avant tout usage public.

Bibliographie et webographie

[1] Spring, Spring Boot Reference Documentation, https://docs.spring.io/spring-boot/

[2] Spring, Spring Security Reference, https://docs.spring.io/spring-security/reference/

[3] Angular, Angular Documentation, https://angular.dev/

[4] FastAPI, FastAPI Documentation, https://fastapi.tiangolo.com/

[5] Qdrant, Qdrant Documentation, https://qdrant.tech/documentation/

[6] Ollama, Ollama Documentation, https://docs.ollama.com/

[7] Qwen Team, Qwen2.5 Technical Report, https://arxiv.org/abs/2412.15115

[8] Unsloth AI, Unsloth Documentation, https://docs.unsloth.ai/

[9] LangChain, LangChain Documentation, https://docs.langchain.com/

[10] OWASP Foundation, OWASP Top 10, https://owasp.org/www-project-top-ten/

[11] IETF, RFC 7519 - JSON Web Token, https://www.rfc-editor.org/rfc/rfc7519

[12] CarQuery API, Vehicle Data API, https://www.carqueryapi.com/

[13] Cloudinary, Image and Video API Documentation, https://cloudinary.com/documentation/

[14] Microsoft, Playwright Documentation, https://playwright.dev/

Annexe A - Cartographie des API

Domaine	Exemples d'opérations
Authentification	register, login, OAuth exchange, recovery request/complete
Annonces	list, detail, create, update, status, delete, my-listings
Images	upload, primary, order, delete
Vendeurs	profile, listings, contact, reviews, reports
Messagerie	inbox, start, detail, send text/image, realtime ticket
Recommandations	feed, settings, update settings, clear history
IA	semantic search, full reindex
Administration	users, seller applications, reports, moderation, audit

Annexe B - Variables de configuration

Variable	Usage
DB_URL / DB_USERNAME / DB_PASSWORD	Connexion MySQL
JWT_SECRET	Signature des jetons d'accès
CORS_ALLOWED_ORIGINS	Origines frontend autorisées
AI_SERVICE_TOKEN	Authentification interne Spring/FastAPI
AI_SERVICE_ENABLED	Activation de la recherche sémantique
OLLAMA_BASE_URL / OLLAMA_MODEL	Accès au modèle local
CLOUDINARY_*	Stockage distant optionnel des images
GOOGLE_* / FACEBOOK_* / MICROSOFT_*	Fournisseurs OAuth optionnels
SMTP_* / TWILIO_*	Récupération de compte hors profil de développement

Règle de gestion
Les fichiers .env, les uploads, les index Qdrant, les caches, les modèles GGUF, les adaptateurs LoRA et les résultats Playwright sont exclus du dépôt Git.

Annexe C - Commandes de validation

Backend

```
cd marketplace-api
.\mvnw.cmd test
```

Frontend

```
cd marketplace-frontend
npm run verify
```

Service IA

```
cd ai-service
.\.venv\Scripts\python.exe -m pytest
```

E2E isolé

```
cd marketplace-frontend
npm run test:e2e
```

E2E pile complète

```
cd marketplace-frontend
npm run test:e2e:live
```

Annexe D Compléments techniques

Fonction	Contrat ou paramètre
Décodage VIN	GET /api/vehicles/vin/{vin} ; authentification ; VIN_DECODER_ENABLED.
Prix indicatif	POST /api/parts/price-estimate ; vendeur ou administrateur.
Identification	POST /api/ai/identify ; fichier authentifié ; relais POST /v1/vision/identify.
Activation vision	VISION_ENABLED ; requirements-vision.txt ; modèle CLIP mis en cache.
Personnalisation	GET /api/recommendations ; GET/PUT /api/recommendations/settings ; DELETE /api/recommendations/history.

La traçabilité technique s'appuie sur PROJECT_HANDOFF.md, ARCHITECTURE.md, RECOMMENDATIONS.md, AGENT_ORCHESTRATION.md, VIN_COMPATIBILITY.md et IMAGE_AND_VOICE_SEARCH.md. Les classes figurées proviennent du paquet entity du backend. Les métriques du modèle et de récupération proviennent respectivement de training/v2-results-120.json et training/retrieval-results.json.